

ArtAdmin - Erfassung kulturhistorischer Objekte

DIPLOMA THESIS

submitted for the

Reife- und Diplomprüfung

at the

Department of Informatik

Submitted by:

Jonas Hinterdorfer
Simon Wögerbauer

Supervisor:

DI David Klewein

Project Partner:

Augustiner-Chorherrenstift St. Florian

Leonding, April 4, 2025

I hereby declare that I have composed the presented paper independently and on my own, without any other resources than the ones indicated. All thoughts taken directly or indirectly from external sources are properly denoted as such.

This paper has neither been previously submitted to another authority nor has it been published yet.

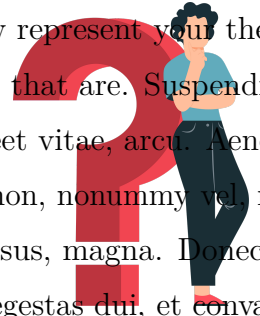
The presented paper is identical to the electronically transmitted document.

Leonding, April 4, 2025

Jonas Hinterdorfer & Simon Wögerbauer

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit, aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Contents

1	Introduction	1
1.1	Project Context and Motivation	1
1.2	Initial Situation and Problem Statement	1
1.3	Objectives and Scope	1
2	Current Situation and Related Work	2
2.1	Institutional and Practical Context	2
2.2	Current Problems and Requirements Derived from Practice	2
2.3	Provenance and Collection-Data Standards	2
2.4	Collection Management Systems and Object Identification	5
2.5	Approaches to 3D Measurement	5
3	Proposed Solution	6
3.1	Solution Overview and Requirements	6
3.2	User Roles and Use Cases	6
3.3	Decision Methodology and Data Model	6
3.4	Technical Architecture Decisions	6
3.5	End-to-End Workflows	6
3.6	Security and Integrity Concept	6
4	Technical Description	7
4.1	System Architecture and Frontend	7
4.2	Backend and Data Model	8
4.3	Data Storage and Search	11
4.4	Image Similarity Search	11
4.5	Depth Camera and 3D Measurement	11
4.6	Authentication, Authorization and Auditability	11
4.7	Deployment and Operations	11
4.8	Testing Concept and Traceability	11

5	Evaluation	12
5.1	Evaluation Methodology and Development Process	12
5.2	Provenance Data Model Evaluation	12
5.3	Depth-Camera Measurement Evaluation	12
5.4	Image Similarity and Duplicate Detection Evaluation	12
5.5	System Functionality and Usability Evaluation	12
5.6	Performance and Operational Evaluation	12
5.7	Lessons Learned and Future Improvements	12
5.8	Limitations and Threats to Validity	12
6	Summary	13
	Bibliography	VI
	List of Figures	VIII
	List of Tables	IX
	List of Listings	X
	Appendix	XI

1 Introduction

1.1 Project Context and Motivation

1.2 Initial Situation and Problem Statement

1.3 Objectives and Scope

2 Current Situation and Related Work

2.1 Institutional and Practical Context

2.2 Current Problems and Requirements Derived from Practice

2.3 Provenance and Collection-Data Standards

One of the main requirements of this system is the import and export of information using established formats. This application stores not only physical properties of objects but also digital ones, such as the history of a specific object (e.g., creation date or when the data entry was created).

After analyzing existing standards, it became clear that no single standard covers all requirements of this project. For this reason, the system combines selected components of several standards. The W3C PROV data model is used to describe the origin and history of the data. Furthermore, LIDO and the Europeana Data Model are added to store cultural-heritage metadata.

The project-specific data schema does not replace these standards. Instead, it combines their core features and extends them where necessary.

2.3.1 W3C PROV and General Metadata Models

The W3C PROV family defines a model for describing the origin and history of any information, but it does not define all metadata required for a collection object. It stores only the provenance.

Provenance is information about entities, activities, and people involved in producing a piece of data or thing, which can be used to form assessments about its quality, reliability or trustworthiness [1].

The W3C PROV family consists of multiple documents; in this system, only the data model is partially used.

The three main concepts of PROV-DM are:

Entity An entity is a physical, digital, conceptual, or other kind of thing with some fixed aspects; entities may be real or imaginary.[2]

Activity An activity is something that occurs over a period of time and acts upon or with entities; it may include consuming, processing, transforming, modifying, relocating, using, or generating entities.[2]

Agent An agent is something that bears some form of responsibility for an activity taking place, for the existence of an entity, or for another agent's activity.[2]

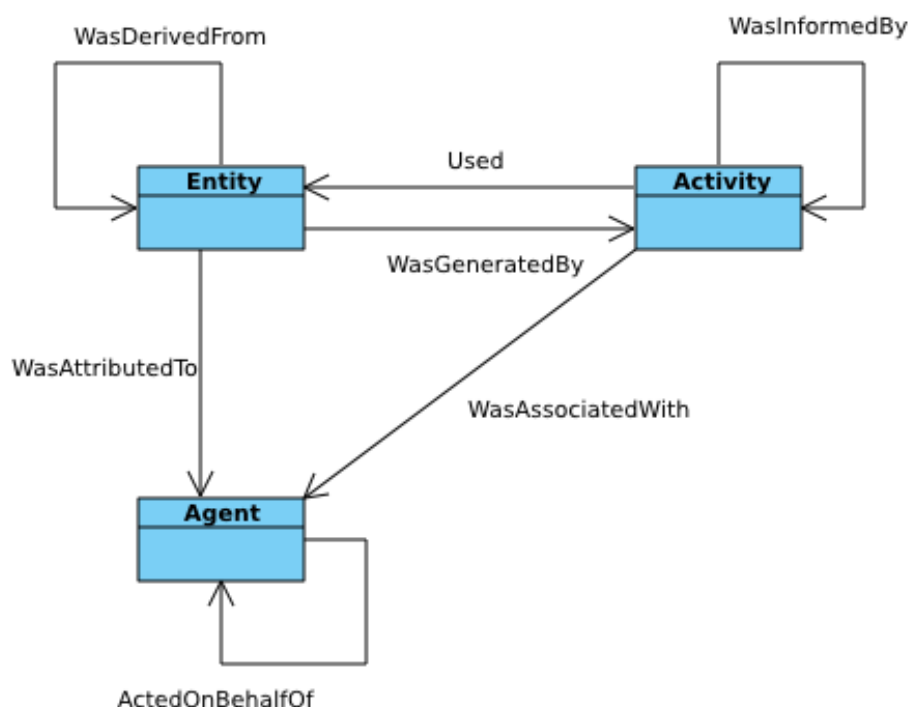


Figure 1: Core concepts of the W3C PROV data model. Source: [2]

2.3.2 CIDOC CRM and Museum-Specific Standards

The CIDOC Conceptual Reference Model is a recognized ISO standard (ISO 21127:2023) designed to store cultural heritage data. According to version 7.1.3, this model contains 99 entity and 198 property declarations [3]. This far exceeds the scope of this project; for that reason, only certain parts are used in the schema of this project. Examples of

the CIDOC CRM concepts used include *E31 Document*, *E39 Actor*, *E53 Place*, and *P108 has produced*.

Another museum-specific standard is LIDO (Lightweight Information Describing Objects), an XML metadata schema for describing objects and works. LIDO is formally defined as an XML harvesting format designed for the exchange of cultural heritage data. The LIDO scheme uses a granular structure but defines only a few required fields to comply with the standard. This low entry barrier allows minimal but compliant records while still supporting detailed descriptions [4].

The data model in ArtAdmin is designed to satisfy the LIDO scheme. By aligning the internal model with this standard, structured import and export into other systems is enabled.

2.3.3 Operational and Linked-Data Standards

To enable large-scale databases that have to work with heterogeneous data, just as Europeana does, a new data schema was created: the EDM (Europeana Data Model). The predecessor, ESE (Europeana Semantic Elements), had multiple issues that were addressed with the EDM. One of the problems ESE had was its flat data model; it consisted of Dublin Core fields that only allowed strings as values. This architecture became overly complex when trying to reference other resources. This is one of the reasons EDM was developed: it solves this problem by adopting an RDF-based graph model that separates entities and linking.

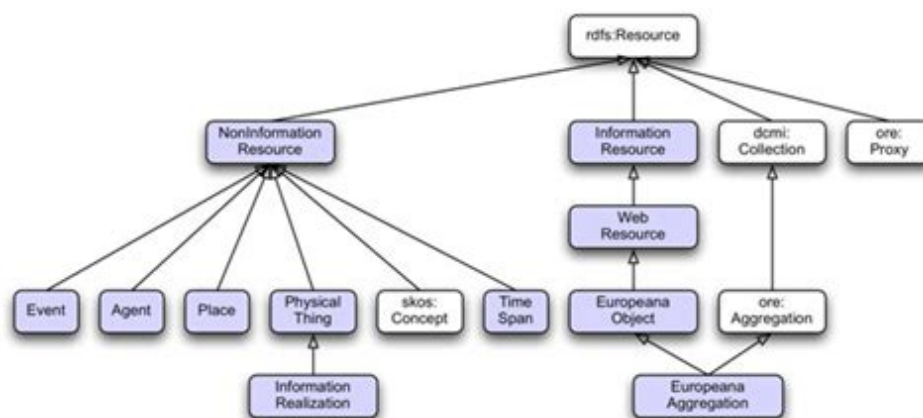


Figure 2: Core concepts of EDM. Source: [5]

One requirement given by our project partner, the Augustiner-Chorherrenstift St. Florian, is that a connection to the Europeana database should be possible. For the

creation of the data model for this project, EDM was not one of the core schemas that were combined because it is less detailed than standards such as LIDO and W3C PROV. However, it is compatible, and the ArtAdmin schema can be mapped to EDM.

2.4 Collection Management Systems and Object Identification

2.4.1 Existing Collection-Management Solutions

2.4.2 Approaches to Object Identification

2.5 Approaches to 3D Measurement

2.5.1 Traditional and Image-Based Methods

2.5.2 Depth-Camera Technologies and Project Hardware

3 Proposed Solution

3.1 Solution Overview and Requirements

3.2 User Roles and Use Cases

3.3 Decision Methodology and Data Model

3.4 Technical Architecture Decisions

3.4.1 Measurement and Similarity Approaches

3.4.2 Storage and Deployment Architecture

3.5 End-to-End Workflows

3.5.1 Object Registration Workflow

3.5.2 Similarity Search Workflow

3.6 Security and Integrity Concept

4 Technical Description

4.1 System Architecture and Frontend

4.1.1 Overall Architecture

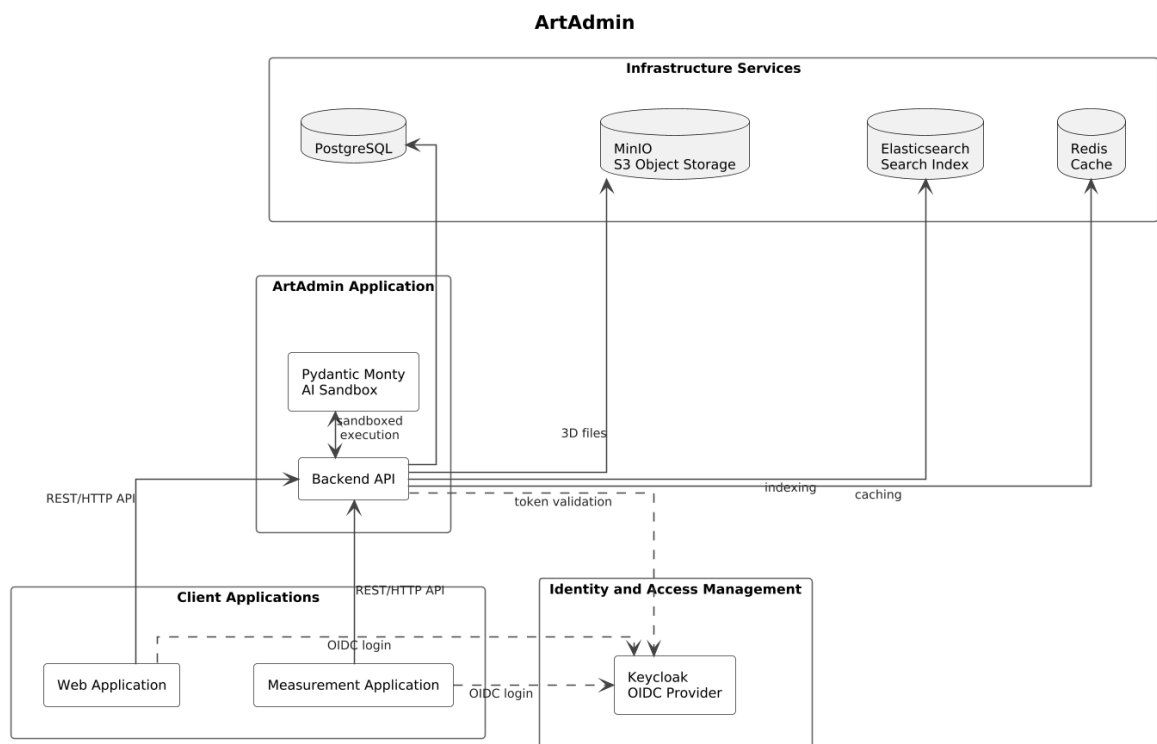


Figure 3: Component diagram of ArtAdmin. Source: [6]

The component diagram 3 shows the overall architecture of ArtAdmin. The system is organized into four layers: the client applications layer, the identity and access management layer, the application layer, and the infrastructure service layer. This separation follows the principle of separation of concerns. The client application isolates the presentation layer from the business logic, while the identity and access management layer centralizes the single point of authentication. Furthermore, the infrastructure layer isolates the services that are needed to run this program. This design also allows for horizontal scaling of the backend, because it is stateless, meaning no information is saved directly within the REST application.

The client applications layer consists of a web application, which serves as the main user interface and handles the administration and display of users and information. It also includes ArtMeasure, a desktop application for calculating object measurements based on a 3D camera. Both applications communicate with the backend via a REST API over HTTP, and all user interfaces authenticate through the same OIDC provider, in this case Keycloak.

The application layer, or backend layer, consists of an ASP.NET Core backend, which is the core program handling all traffic and communicating with all services. Furthermore, this layer includes Pydantic Monty, a secure Python interpreter written in Rust for executing code generated by AI agents [7]. This software is used to prevent ArtPilot from executing unauthorized and unsafe code while still enabling the import of any format.

This software solution uses four additional services, all of which can be self-hosted. For storing structured information, we use PostgreSQL. For image and vector data storage, we use MinIO, because it implements the S3 API and can therefore be replaced at any time by another program that also implements this API, such as AWS S3. Furthermore, for the use case of searching information, we use Elasticsearch, because it provides the speed and full-text search capabilities required. Redis is used as a cache for request results to speed up the application; it was chosen because it is a fast, in-memory key-value store, which is a widely used choice for this use case. [8]

4.1.2 Frontend Implementation

4.2 Backend and Data Model

4.2.1 Choice of Backend Technology [9]

For the backend of ArtAdmin, C# with ASP.NET Core was chosen over alternative technology stacks such as Java (Spring Boot, Quarkus), Go, and TypeScript (NestJS). The requirements for this project were that the technology be strongly typed and maintainable, and that it integrate seamlessly with the planned architecture, including Entity Framework Core, PostgreSQL, Elasticsearch, MinIO, Redis, Keycloak, and .NET Aspire. [6]

The following table compares ASP.NET Core with Spring Boot, Quarkus, Gin (Go), and NestJS (TypeScript) with respect to the criteria relevant to ArtAdmin.

Table 1: Comparison of backend technology candidates for ArtAdmin.

Performance: TechEmpower Round 23, Fortunes test, PostgreSQL, requests per second (req/s), rank among 523 tests [10].

Technology	Typing	Performance (Fortunes, R23)	Resource usage	Relevant characteristics
C# / ASP.NET Core	Static, object-oriented	Best entry: aspnetcore, 741,878 req/s (rank 35) Full-stack: aspnetcore-mvc, 322,802 req/s (rank 122) <i>Analysis:</i> reference values for the comparison.	Not evaluated in the selected Spring Boot–Quarkus study.	Built-in dependency injection and middleware support [11].
Java / Spring Boot	Static, object-oriented	Best entry: spring-webflux-r2dbc, 245,588 req/s (rank 177) Full-stack: spring, 243,639 req/s (rank 179) <i>Analysis:</i> 33 % (best) and 75 % (full-stack) of the ASP.NET Core values.	Higher startup time, CPU and RAM usage than Quarkus in the selected study [?].	Large, established Java ecosystem.
Java / Quarkus	Static, object-oriented	Best entry: quarkus, vert.x, 1,028,408 req/s (rank 14) Full-stack: quarkus, 431,464 req/s (rank 89) <i>Analysis:</i> 1.39× (best) and 1.34× (full-stack) the ASP.NET Core values.	Lower startup time, CPU and RAM usage than Spring Boot in the selected study [?].	Supports JVM and GraalVM native executable deployments [12].
Go	Static, compiled	Best entry: fasthttp-prefork, 959,399 req/s (rank 17) Full-stack: goravel fiber, 549,130 req/s (rank 62) <i>Analysis:</i> 1.29× (best) and 1.70× (full-stack) the ASP.NET Core values.	Not evaluated in the selected resource-usage study.	The standard library provides HTTP client and server implementations [13].
TypeScript / NestJS	Statically typed source code, transpiled to JavaScript	Best entry: nestjs-fastify, 160,502 req/s (rank 230) Default entry: nestjs, 82,444 req/s (rank 321) <i>Analysis:</i> 22 % (best) and 26 % (default vs. full-stack) of the ASP.NET Core values. No full-stack-class NestJS entry exists.	Not evaluated in the selected resource-usage study.	Uses modules, providers and dependency injection [14].

The comparison shows that all considered technologies are technically capable of implementing the ArtAdmin backend. Go and Quarkus achieve higher throughput in the selected TechEmpower benchmark, while Quarkus also provides advantages for applications that require very short startup times or a low memory footprint. However,

these advantages are mainly relevant for serverless applications, edge deployments, or a large number of small services that are started frequently. ArtAdmin is designed as a long-running, containerized REST application. Its main workload consists of communication with PostgreSQL, MinIO, Elasticsearch, Redis, and Keycloak. Therefore, the maximum HTTP throughput of the framework is not the main limiting factor for the expected workload.

Quarkus was considered the main alternative to ASP.NET Core because it is a modern Java framework with support for REST APIs, dependency injection, containerized development, and integrations for the required infrastructure services. The benchmark results show that Quarkus can achieve higher throughput than ASP.NET Core in the selected test scenario. Nevertheless, the advantages of Quarkus in startup time and memory consumption are most significant when the application is compiled as a GraalVM native executable. Native compilation requires additional build time and may require explicit configuration of reflection and dynamic class loading [12, 15]. These additional requirements are not necessary for ArtAdmin because the backend is intended to run continuously as a containerized service.

The developer team has experience in both Java and C#. However, through a previous project and the focus of the previous school year, the team has more extensive practical knowledge of C# and the .NET stack. This includes the development of REST APIs, database applications, Docker-based deployments, Kubernetes environments, and authentication with Keycloak, PostgreSQL, and Elasticsearch. By choosing C#, the implementation risk was reduced, which allows the team to focus on project requirements such as the data model and other features. Existing skills and project-specific constraints are relevant factors when selecting a programming language ecosystem [16]. For these reasons, C# was selected as the backend technology for this project. This decision does not imply that Quarkus, Go, or TypeScript-based frameworks are technically unsuitable, but in this use case, given the experience of the developer team, .NET is the best choice.

4.2.2 Object and Provenance Data Model

4.2.3 ArtPilot Architecture

4.3 Data Storage and Search

4.3.1 Database and Media Management

4.3.2 Full-Text Search and Filtering

4.4 Image Similarity Search

4.4.1 Processing Pipeline

4.4.2 Similarity Computation and Robustness

4.5 Depth Camera and 3D Measurement

4.5.1 Hardware, Software and Measurement Principle

4.5.2 Data Quality and Measurement Uncertainty

4.5.3 Measurement versus Scanning and Limitations

4.6 Authentication, Authorization and Auditability

4.7 Deployment and Operations

4.8 Testing Concept and Traceability

5 Evaluation

5.1 Evaluation Methodology and Development Process

5.2 Provenance Data Model Evaluation

5.3 Depth-Camera Measurement Evaluation

5.3.1 Experimental Setup and Metrics

5.3.2 Results and Interpretation

5.4 Image Similarity and Duplicate Detection Evaluation

5.4.1 Dataset and Metrics

5.4.2 Threshold Evaluation and Practical Usability

5.5 System Functionality and Usability Evaluation

5.6 Performance and Operational Evaluation

5.7 Lessons Learned and Future Improvements

5.8 Limitations and Threats to Validity

6 Summary

Aufzählungen:

- Itemize Level 1
 - Itemize Level 2
 - Itemize Level 3 (vermeiden)
- 1. Enumerate Level 1
 - a. Enumerate Level 2
 - i. Enumerate Level 3 (vermeiden)

Desc Level 1

Desc Level 2 (vermeiden)

Desc Level 3 (vermeiden)

Bibliography

- [1] P. Groth and L. Moreau, “PROV-overview: An overview of the PROV family of documents,” World Wide Web Consortium, W3C Working Group Note, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-overview/>
- [2] L. Moreau and P. Missier, “PROV-DM: The PROV data model,” World Wide Web Consortium, W3C Recommendation, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>
- [3] C. Bekiari, G. Bruseker, E. Canning, M. Doerr, P. Michon, C.-E. Ore, S. Stead, and A. Velios. (2024) Definition of the CIDOC conceptual reference model. Version 7.1.3. Accessed: 2026-09-23. [Online]. Available: https://cidoc-crm.org/sites/default/files/cidoc_crm_version_7.1.3.pdf
- [4] J. Rössel. (2025) LIDO – ein standard, viele kulturobjekte [webinar]. [Online]. Available: <https://info.kulturpool.at/lido-ein-standard-viele-kulturobjekte/>
- [5] A. W.-C. Huang. (2013, Sep.) Activating linked open data in libraries, archives and museums. [Online]. Available: <https://andrea-index.blogspot.com/2013/09/activating-linked-open-data-in.html>
- [6] J. Hinterdorfer. (2026, Sep.) Support in PlantUML design. [Online]. Available: <https://www.perplexity.ai/search/37034c3b-f743-488e-96ab-ab603ba456c4>
- [7] S. Colvin. (2026, Feb.) Pydantic monty: You probably don’t need a full sandbox. [Online]. Available: <https://pydantic.dev/articles/pydantic-monty>
- [8] J. Hinterdorfer. (2026, Sep.) Architecture cleanup. [Online]. Available: <https://www.perplexity.ai/search/a5e84791-a73e-4c07-ac79-51ac6252d630>
- [9] ——. (2026, Sep.) Cleanup of the topic: Choice of backend technology. [Online]. Available: <https://www.perplexity.ai/search/a32d3349-3212-4d30-afb9-dba965ecd02e>
- [10] TechEmpower, “Web framework benchmarks, round 23,” <https://www.techempower.com/benchmarks/#section=data-r23&test=fortune>, 2025, performance comparison of community-contributed framework implementations.
- [11] Microsoft, “Dependency injection in ASP.NET Core,” <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection>, 2026.
- [12] Quarkus, “Building a native executable,” <https://quarkus.io/guides/building-native-image/>, 2026.
- [13] The Go Authors, “Package net/http,” <https://pkg.go.dev/net/http>, 2026.
- [14] NestJS, “Fundamentals,” <https://docs.nestjs.com/>, 2026.
- [15] Quarkus, “Tips for writing native applications,” <https://quarkus.io/guides/writing-native-applications-tips/>, 2026.

- [16] M. Deldar *et al.*, “A decision model for programming language ecosystem selection: Seven industry case studies,” *Information and Software Technology*, vol. 135, p. 106578, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584921001051>

List of Figures

1	Core concepts of the W3C PROV data model. Source: [2]	3
2	Core concepts of EDM. Source: [5]	4
3	Component diagram of ArtAdmin. Source: [6]	7

List of Tables

1	Comparison of backend technology candidates for ArtAdmin. <i>Performance:</i> TechEmpower Round 23, Fortunes test, PostgreSQL, requests per second (req/s), rank among 523 tests [10].	9
---	--	---

List of Listings

Appendix